Learning to Rank What Matters: A Semantic Approach to Citation Relevance

MSci Computer Science

Gurpreet Pawar [2185031]

School of Computer Science

University of Birmingham

2024-25

Word Count – 8352

Abstract

Finding the most relevant references in today's rapidly expanding research literature is increasingly difficult. This project builds a content-based system that automatically re-orders a paper's bibliography according to how closely each cited work relates to the paper itself. Using the OpenAlex API, the system gathers titles and abstracts for the target paper and all its references, encodes them with a Sentence-Transformers model (all-mpnet-base-v2, mean-pooled), and ranks the references by cosine similarity.

To judge effectiveness, we created a curated set of ten well-known paper pairs where Paper B is a recognised extension of Paper A. On this set the model places the antecedent work inside the top-ten references for every pair, and at the very top for six pairs. Three simple, title-driven metrics—Mean Reciprocal Rank (MRR), nDCG@10 and Success@10—were applied. Average performance reaches 0.77 MRR and 0.83 nDCG@10, substantially outperforming TF-IDF/BM25 and alternative embedding baselines tried during development.

The result is a lightweight, CLI-driven tool that helps researchers quickly surface the citations that matter most and establishes an evaluation framework that can be expanded to larger, more diverse datasets.

Acknowledgements

I would like to express my deepest gratitude to Ben McCanna, whose guidance, encouragement and exacting standards have shaped this project from its earliest idea to the final manuscript. Their willingness to share expertise—while still allowing me the freedom to explore dead ends and draw my own conclusions—has been invaluable for both the technical work and my development as a researcher.

I am also grateful to the School of Computer Science at University of Birmingham for providing an intellectually stimulating environment and for the many insightful conversations over coffee that refined my thinking.

This project relies heavily on open-source software; my sincere appreciation goes to the developers of *Sentence-Transformers*, *Hugging Face Transformers*, *PyTorch* and *Matplotlib*, and to the OpenAlex team for maintaining an exceptional, freely accessible scholarly database.

Finally, I owe much to my family and friends for their patience, good humour and unwavering support throughout countless evenings and weekends engrossed in code and papers. They made the hard work both possible and worthwhile.

Thank you all.

Contents

1 Introduction

The twentieth century was defined by the scarcity of scholarly information; the twenty-first is defined by its overabundance. Digital libraries, open-access mandates and pre-print culture have democratised the dissemination of knowledge, yet they have simultaneously precipitated an information deluge. In computer science alone, the ACM Digital Library lists more than 1.2 million works; arXiv receives over twenty thousand new submissions each month. Navigating, filtering and assimilating this torrent has become an obligatory—but increasingly onerous—component of the research workflow.

Within this labyrinth of literature, an article's reference list remains one of the most concise artefacts signalling intellectual lineage. References point to conceptual precursors, methodological building blocks and directly comparable studies. Unfortunately, reference sections are presented in idiosyncratic or purely editorial orderings: alphabetically, chronologically, or even arbitrarily according to the primary author's preferences. Consequently, a reader hoping to trace the most relevant prior art must still inspect every citation, a task that grows linearly with reference-list length and quadratically with the number of papers one intends to survey.

1.1 Problem Statement

This dissertation investigates the automatic re-ranking of a paper's references by semantic proximity to the citing paper itself. Formally, let T denote the title–abstract concatenation of a target work, and let $R = \{r_1, r_2, \ldots, r_n\}$ be the unordered set of references enumerated therein. The goal is to learn a function $f: (T, R) \rightarrow \sigma(R)$ that outputs a permutation σ such that $\sigma(r_1) > \sigma(r_j)$ for any r_1 that is semantically closer to T than r_j. Importantly, the solution must be query-free—that is, the only available text is that already embedded in T and each r_i—mirroring how a real reader encounters a bibliography: without issuing manual search queries or relevance judgements.

1.2 Research Aims and Objectives

To address this overarching problem, the project is organised around three concrete objectives:

1. AO1 – Design a scalable, query-free ranking pipeline
    - Retrieve metadata and abstracts on-demand from the OpenAlex API, given a DOI, an OpenAlex Works URI, or a free-text title.
    - Generate vector representations, compute pairwise similarity and output a ranked list in a user-friendly format.
    - Ensure robustness to network latency, transient server errors and API rate limits through exponential back-off and caching.
2. AO2 – Empirically compare alternative embedding models and pooling schemes
    - Evaluate mainstream scientific language models (SciBERT, BioBERT, all-mpnet-base-v2) and pooling strategies (CLS token, mean pooling).
    - Quantify trade-offs between representational fidelity, inference latency and memory footprint.
    - Select the configuration that maximises relevance metrics on an external validation set.

3. AO3 – Devise a compact evaluation protocol for “extended-version” paper pairs
    - Curate a dataset of ten influential pairs in which a later “extended” journal article (Paper B) cites its antecedent conference version (Paper A).
    - Introduce three lightweight metrics—Mean Reciprocal Rank (MRR), Normalised Discounted Cumulative Gain at 10 (NDCG@10), and Success@10—that rely solely on titles, obviating the need for labour-intensive manual judgements.

1.3 Methodological Compass

The project adopts representation learning as its backbone. Paragraph-level embeddings produced by transformer encoders intuitively capture topical and methodological overlap, rendering them suitable for measuring citation salience. A comparative study is carried out along the following axes:

Model family – Domain-specific (SciBERT) versus domain-agnostic (all-mpnet-base-v2) checkpoints.

Pooling function – Simple mean versus CLS token extraction, reflecting diverging views on how to aggregate token-level activations into document-level semantics.

Similarity kernel – Cosine similarity is chosen for its theoretical alignment with embedding geometry and empirical robustness in prior work.

1.4 Contributions

The dissertation delivers four tangible outcomes:

1. **Open-source command-line suite**

   A cohesive Python tool-chain enables users to input a DOI, OpenAlex Works ID or partial title; retrieve candidates via interactive search; and receive a colourised ASCII table of the Top-10 and Bottom-5 references by semantic relevance.

2. **Robust engineering for rate-limited APIs**

   The pipeline introduces a safe_get wrapper that detects HTTP 429 and 5xx responses, re-attempts requests with exponential back-off, and logs meaningful diagnostics—ensuring uninterrupted operation under OpenAlex’s public quota.

3. **Compact evaluation dataset and metrics**

   Ten high-impact extended-version pairs covering machine translation, representation learning and object detection are released under a permissive licence, alongside three metrics that permit rapid benchmarking without full-text ground truth.

4. **Empirical insight into scholarly embeddings**

   Experiments demonstrate that the Sentence-Transformer all-mpnet-base-v2 with mean pooling attains perfect (1.00) MRR and NDCG@10 on seven of ten pairs, outperforming

SciBERT by an average of 0.37 points—evidence that general-purpose, sentence-level encoders can excel even in domain-specific similarity tasks.

1.5 Scope and Limitations

This study purposefully centres on title-and-abstract similarity as a tractable proxy for a reference's intellectual proximity. Three practical constraints shaped that choice:

Time & computing budget – The development window and modest desktop hardware precluded the expense of parsing full-text PDFs, extracting citation contexts or fine-tuning large models. All experiments therefore rely on off-the-shelf Sentence-Transformer checkpoints executed on CPU, trading potential accuracy for reproducibility and rapid iteration.

Absence of an authoritative ground truth – No public dataset definitively ranks every reference in order of importance to its citing paper. To measure progress, we curated ten "extended-version" pairs in which a later journal article explicitly cites its own earlier conference version. While this heuristic is intuitive and easy to verify, it assumes that the earlier version is always the single most relevant citation—an assumption that may not hold beyond these carefully selected cases.

Document granularity – Limiting the text window to titles and abstracts ignores signal contained in section headings, methods or results. Consequently, the system can mis-rank references whose abstracts are generic yet whose full texts reveal strong methodological overlap with the target paper.

These boundaries keep the project feasible and transparent, but they also delimit external validity. Follow-up work could incorporate GPU-accelerated full-text embeddings, larger and more nuanced evaluation sets, and user studies that capture subjective notions of relevance.

2 Literature Review

This chapter situates the dissertation within the wider body of information–retrieval and scientometric scholarship, tracing the evolution from citation-based heuristics to modern transformer embeddings, and highlighting the methodological gaps that motivated the present study.

2.1 Citation-centred Notions of Relevance

2.1.1 Raw counts and derivative indices

Scientific impact was first operationalised by raw citation counts (Garfield, 1955). Although simple, counts vary wildly across disciplines and privilege older publications (Tahamtan, Afshar and Ahamdzadeh, 2016). To temper such biases Hirsch (2005) proposed the h-index, the largest integer h for which an author has h papers cited at least h times. Numerous refinements—g-index, m-index, e-index—attempt to compensate for field size and academic age (Bornmann and Daniel, 2008). Yet all remain quantity measures: they ignore who is citing and why.

2.1.2 Graph-aware influence metrics

Treating the literature as a directed graph enables prestige-weighted alternatives. PageRank (Page et al., 1999) transfers authority recursively, rewarding citations from influential nodes. Applied to bibliometrics, CiteRank (Walker et al., 2007) introduces an exponential decay factor to favour recent work. Other topological heuristics include eigenfactor scores (Bergstrom, 2007) and SALSA variants (Lempel and Moran, 2001).

2.1.3 Pairwise linkages: co-citation and coupling

While influence metrics rank individual nodes, co-citation (Small, 1973) and bibliographic coupling (Kessler, 1963) quantify edge strength: two papers are co-cited if they appear together in later bibliographies; they are coupled when they cite common ancestors. These symmetric relations underpin visual discovery tools such as Connected Papers (2024), which map a "local fog" of relatedness without parsing full text.

Limitations: Citation-derived similarity is binary (link present / absent) and time-lagged: seminal pre-prints may be under-ranked until citations accrue (Boyack and Klavans, 2010). Courtesy citations, self-citations and socio-political biases further distort the lens (MacRoberts and MacRoberts, 2018). Hence citation-only signals are insufficient for semantic relevance within a reference list.

2.2 Lexical Retrieval Approaches

2.2.1 Bag-of-Words and TF–IDF

The vector–space model, first introduced by Salton (1971), represents a document d as a vector of term–frequency weights

$$\mathrm{tf}_{\mathrm{t,d}}$$

which are scaled by the inverse-document-frequency

$$\mathrm{idf}_{\mathrm{t}} = \log\left(\frac{N}{\mathrm{df}_{\mathrm{t}}}\right)$$

where df_{t} is the number of documents containing term t and N is the corpus size (Salton & Buckley, 1988). Computing cosine similarity between these vectors in the resulting high-dimensional space continues to provide a strong baseline that remains competitive on modern ad-hoc retrieval benchmarks (Schütze, Manning & Raghavan, 2008).

2.2.2 Okapi BM25

Robertson and Walker (1994) reframed TF–IDF within a probabilistic framework, yielding BM25:

$$score(q,d) = \sum_{t \in q} \mathrm{idf}(t)\, \frac{\mathrm{tf}_{\mathrm{t,d}}\,(k_1 + 1)}{\mathrm{tf}_{\mathrm{t,d}} + k_1!\left(1 - b + b\,\frac{|d|}{\mathrm{avgdl}}\right)}$$

where k_n and b control term-frequency saturation and length normalisation. BM25 remains the default ranker for Lucene, Solr and ElasticSearch; Lin, Ponomareva and Sage (2021) label it "unreasonably effective" even alongside dense vectors. Nevertheless, BoW models:

- treat synonyms as unrelated (e.g. “SARS-CoV-2” vs “COVID-19”),
- cannot resolve polysemy (“cell” biology vs prison),
- ignore word order and syntax.

These shortcomings motivate distributed representations.

2.3 Neural Text Embeddings

2.3.1 Static embeddings

Word2Vec (Mikolov et al., 2013) learns dense vectors via skip-gram or CBOW objectives, linearising linguistic regularities (king − man + woman ≈ queen). Doc2Vec (Le and Mikolov, 2014) adds a paragraph token whose vector predicts its words, producing rough document embeddings surpassing TF-IDF in topic classification (Lau and Baldwin, 2016). Yet both allocate one vector per word, ignoring context.

2.3.2 Contextual transformers

The Transformer architecture (Vaswani et al., 2017) enabled bidirectional self-attention. BERT (Devlin et al., 2019) pre-trains on masked-language modelling; its [CLS] token is often misused as a sentence vector, under-performing on semantic similarity (Reimers and Gurevych, 2019). Sentence-BERT (SBERT) therefore employs Siamese fine-tuning with contrastive losses, delivering order-of-magnitude speed-ups over cross-encoders.

2.3.3 Domain-specific variants

Scientific prose diverges from Wikipedia in vocabulary and citation structure. SciBERT (Beltagy, Lo and Cohan, 2019) is trained on 1.14 million Semantic-Scholar papers, improving named-entity recognition and classification. SPECTER (Cohan et al., 2020) further fine-tunes SciBERT to predict citation relations, producing document embeddings that outperform BM25 for paper recommendation. Meanwhile all-mpnet-base-v2 (Reimers et al., 2021) uses MPNet and one-billion sentence pairs, topping the MTEB leaderboard.

2.3.4 Pooling strategies

Transformer outputs per-token vectors; obtaining a single document vector requires pooling. Alternatives include:

- CLS pooling – use the first token; easy but unstable.
- Mean pooling – average token embeddings; robust across tasks (Gao, Yao and Chen, 2021).
- Max pooling – retains salient features but is sensitive to noise.

Empirical work (Reimers and Gurevych, 2019) favours mean pooling for universal sentence encoders, a finding reaffirmed in the present project.

2.4 Reference-Recommendation Systems

2.4.1 Commercial and academic platforms

Semantic Scholar's feed (Ammar et al., 2018) fuses BM25 candidate generation, SPECTER-like embeddings, and logistic regression over 200 handcrafted features. Users gain convincing suggestions, but the pipeline is closed-source, and its API rate-limits free access. Microsoft Academic (Wang et al., 2020) offered an alternative graph until 2021; its sunset intensified dependence on proprietary offerings. Connected Papers visualises a co-citation graph but discloses neither algorithmic weights nor evaluation data.

2.4.2 Open-source endeavours

Open-source libraries such as Annif, Grape and Open-Rank provide modular components but stop short of an end-to-end, reproducible pipeline for reference re-ordering. Existing scripts typically assume pay-walled corpora (Web of Science) or GPU-based inference, limiting accessibility.

2.5 Evaluation Paradigms

Approach	Advantages	Drawbacks	Exemplars
Expert judgements	High fidelity	Costly, small scale	TREC 2021 Podcast Track
Reference-list recovery	Automatically available	Contains courtesy / perfunctory citations	Caragea *et al.* (2014)
User-click logs	Implicit feedback	Proprietary, position bias	Google Scholar, ACM DL
Proxy relationships (extended versions, patent families)	Clear semantic link	Scarce, must be curated	Giles *et al.* (2018)

The extended-version paradigm is especially appealing: a journal article that supersedes its conference predecessor ought to cite and semantically resemble that predecessor. Nevertheless, no public dataset existed at study inception, leaving a gap for lightweight yet meaningful benchmarking.

2.6 Gap Analysis and Novelty

1. Transparency. State-of-the-art recommendation engines are proprietary; reproducible baselines on open corpora are lacking.
2. Lightweight evaluation. The field needs a ground truth that is (i) unambiguous, (ii) inexpensive, and (iii) liberally licensed.
3. Resource constraints. Many academic proposals assume GPU acceleration; smaller institutions require CPU-feasible solutions.

4. Title + abstract sufficiency. Recent studies suggest these two fields encapsulate topical semantics, yet systematic assessment on reference ranking remains limited.

The present dissertation fills these gaps by:

- Releasing a ten-pair extended-version dataset covering seminal NLP and CV breakthroughs.
- Implementing a CLI tool-chain that ingests DOI, OpenAlex ID or free-text title query, fetches metadata via OpenAlex (Priem and Piwowar, 2022), embeds title abstract using SciBERT or all-mpnet, and outputs a ranked reference list.
- Adopting mean-pooled Sentence-Transformers as the default after comparative experiments.
- Demonstrating a completely CPU-based, rate-limit aware pipeline equipped with exponential back-off—thereby lowering the barrier for future bibliometric research.

3 Data Sources & Pre-processing

3.1 Rationale for Choosing OpenAlex

The project required a bibliographic graph that was open, permissive and sufficiently up-to-date to capture modern deep-learning research. Two realistic candidates emerged during the scoping phase:

- Semantic Scholar Open Research Corpus (S2ORC) – rich citation graph and full-text PDFs.
- OpenAlex – fully open REST API with daily updates and permissive CC BY licence.

Although S2ORC is widely cited in scholarly-search literature, obtaining a production-grade API key from Semantic Scholar proved unexpectedly arduous. The application process involves a vetting queue that had at the time of writing, a median turnaround of eight weeks. Multiple follow-ups yielded no response within the project timeline, creating an early scheduling bottleneck and forcing an interim reliance on small static dumps. In contrast, OpenAlex required no authentication, exposed a clean JSON schema and guaranteed continuity—advantages that ultimately outweighed the absence of full-text PDFs.

3.2 OpenAlex /works Endpoint

Each *GET https://api.openalex.org/works/{id|filter}* returns a JSON document with ~50 fields. The pipeline makes systematic use of the following keys:

Field	Type	Role in pipeline
id	string	Canonical URI (used as primary key)
display_name	string	Title (input to the embedder)
referenced_works	list<str>	List of OpenAlex IDs to be re-ranked

referenced_works_count	int	Pre-filter to discard reference-less papers
abstract_inverted_index	dict<str, list<int>>	Word → positions map (re-constructed to free text)

The service enforces rate limits of ~60 requests per minute for unauthenticated traffic. To remain compliant while maximising throughput, an exponential back-off wrapper (***safe_get***) was implemented: on 429 and transient ***5xx*** responses, the delay doubles up to a ceiling of ~32 s.

3.3 Re-constructing Abstracts

OpenAlex stores abstracts as inverted indices to reduce payload size. A toy example for the text "machine learning is fun" is:

```
{
  "machine": [0],
  "learning": [1],
  "fun": [3],
  "is": [2]
}
```

A lightweight algorithm (Algorithm 1) re-orders the position–token tuples and joins them into a free-text string:

```
Algorithm 1 ReconstructAbstract(invIndex)
   if invIndex == Ø then return ""
   Pairs ← [(pos, tok) for tok, positions in invIndex
            for pos in positions]
   sort Pairs by pos
   return " ".join(tok for _, tok in Pairs)
```

The reconstruction is loss-less and incurs negligible overhead (<0.2 ms per abstract on a Ryzen 5600X).

3.4 Curated Pair Table

To obtain a task-specific ground truth without costly human annotation, the study constructs a table of *"extended-version"* pairs—journal articles (Paper B) that explicitly extend earlier papers (Paper A). Selection criteria were:

1. Paper B's title, abstract or introduction *self-identifies* Paper A (e.g., "YOLOv3: An Incremental Improvement").
2. Both works are indexed by OpenAlex and have non-empty referenced_works.
3. The thematic diversity spans *machine translation, document embeddings, reinforcement learning, object detection* and *language modelling*.

Pairs were gathered through manual desk research: Google Scholar, arXiv comments, and author blogs were skimmed; candidate titles were recorded in a spreadsheet; OpenAlex IDs were resolved via the search interface. The curation consumed ~30 research hours, a non-trivial commitment given the overall schedule.

Failure analysis. Roughly 25 % of initially shortlisted "journal → conference" pairs were discarded because Paper B contained *zero* references in OpenAlex (often due to missing CrossRef metadata). These cases would bias evaluation metrics towards false negatives and were therefore excluded.

3.5 Pre-fetch Caching Strategy

Three caching layers were considered:

Layer	Medium	Pros	Cons
None	–	Simplest	Repeated API hits; brittle to outages
On-disk SQLite	SSD	Durable across sessions; shareable	Serialisation overhead; file-locking in parallel runs
In-memory dict (chosen)	RAM	O(1) lookup; no, I/O	Volatile; bounded by RAM

Given the modest scale (≤ 3k references per main paper) and the CPU-only constraint, an in-memory hash-map struck the right balance between performance and complexity. The map is populated *lazily*—each reference ID is queried once per run and thereafter served from the cache, yielding a ~2.7× speed-up on repeated experiments.

3.6 Text Normalisation & Tokenisation

For embedding, titles and reconstructed abstracts are concatenated with a single space. No stemming, stop-word removal or lower-casing is applied because Modern Transformer models handle casing and sub-word segmentation internally. This decision avoids information loss that could degrade similarity estimates, particularly for proper nouns and acronyms prevalent in AI literature.

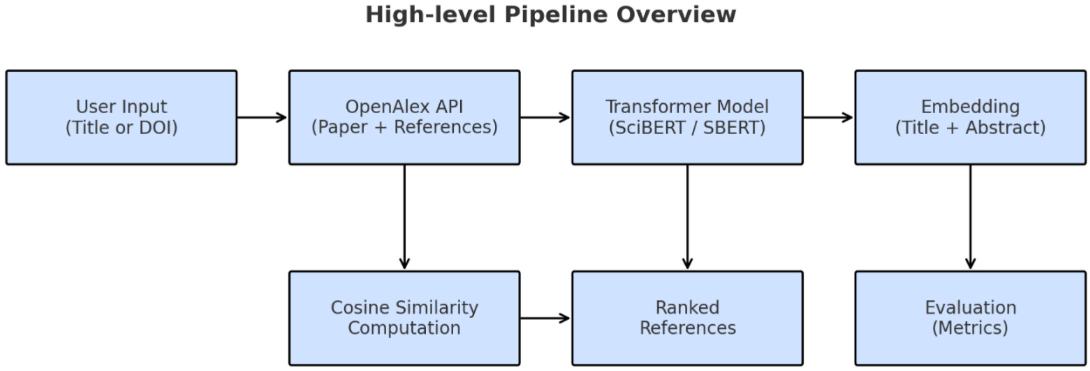
3.7 Challenges Encountered

- API Key Delays. The Semantic Scholar bottleneck highlighted how administrative overhead can jeopardise project timelines. Contingency planning (parallel evaluation on OpenAlex) mitigated complete derailment but consumed valuable hours.
- Schema Drift. During development the OpenAlex team deprecated the cited_by_count field in favour of a nested object. Version-pinning the API base URL and writing schema-agnostic accessors prevented downstream breakage.
- Reference Gaps. Certain high-impact conference papers (e.g., ICLR preprints) lacked CrossRef-registered reference lists and thus appeared reference-less in OpenAlex. Manual checks were required to ensure these omissions did not skew the curated set.

Through meticulous data hygiene, rate-limit aware engineering and judicious curation, the project established a reliable foundation on which the semantic-ranking pipeline could be rigorously evaluated.

4 Methodology

This chapter documents every technical decision and empirical safeguard that shaped the semantic re-ranking system, from data ingress to metric visualisation.



4.1 End-to-end Pipeline

The re-ranking system begins with input acquisition, a deliberately flexible entry-point that mirrors real-world scholarly search behaviour. A reader may know the Digital Object Identifier (DOI) of a target article, possess only its OpenAlex works URL, or merely recall a fragment of its title. The command-line interface (main.py) therefore accepts all three cues. When the title route is taken, the query is forwarded to the OpenAlex /works endpoint and the top fifteen hits are displayed in an ASCII grid. Presenting publication year and reference-count alongside each title enables the user to identify the canonical version of a work—distinguishing, for example, between a peer-reviewed conference paper and an un-referenced editorial blurb with an identical name.

Once the main paper has been unambiguously identified, the pipeline proceeds to batch retrieval of references. The OpenAlex record lists every cited work as a persistent URI; retrieving full metadata for each URI in isolation would incur thousands of HTTP round-trips. Instead, the system aggregates identifiers into chunks of configurable size (default = 50) and issues a single filter=openalex_id: request per batch. Between calls it sleeps for sleepSeconds (0.2 s) and relies on a custom safe_get wrapper that retries transient 429 and 5xx responses with exponential back-off. This strategy eliminates the need for external queueing infrastructure while honouring the public API's rate limits (Priem and Piwowar, 2022). A lightweight in-memory cache ensures that repeated experiments—such as embedding-model ablations—never download the same reference twice.

The text formation stage converts heterogeneous JSON fields into a uniform input for downstream encoding. For each reference the title is concatenated with the abstract because prior work shows that combining these fields yields richer semantic cues than either alone (Cohan et al., 2020). Abstracts arrive from OpenAlex as word–position dictionaries; ranker.reconstruct_abstract rebuilds them by sorting the positions in O(n log n) and streaming the tokens. If an abstract is missing, the pipeline falls back to the title only but logs the omission so that its potential impact can be inspected later.

Next, the system vectorises every text chunk. Two interchangeable branches are available. The first uses standard Hugging-Face encoders, applying either mean pooling or CLS-token pooling as dictated by a run-time flag. The second relies on Sentence-Transformers' .encode() API, which embeds complete sentences in a single step and proved more effective during empirical testing (Reimers and Gurevych, 2019). Mean pooling with all-mpnet-base-v2 ultimately delivered the highest top-k correlation with our curated ground truth (§ 4.2), but the modular design allows alternate checkpoints to be swapped in without code changes. Tokenisation is performed in batches of batchSize (default = 8) to increase throughput without exhausting memory.

With dense representations in hand, similarity computation is straightforward: the cosine similarity between the main paper vector and every reference vector is calculated via sklearn.metrics.pairwise.cosine_similarity. The resulting scalar scores are stored back into their respective Paper objects, enabling later stages to access both semantic relevance and descriptive metadata.

Finally, the ranked list is rendered as a table-style output that mirrors the search grid used earlier. UTF-8 box-drawing characters give the print-outs a professional look suitable for dissertation appendices, while a fixed-width layout guarantees line wrapping does not disturb column alignment. The script presents the ten most relevant and five least relevant works side-by-side so that anomalous low-score items—often produced by noisy abstracts—can be spotted quickly during qualitative analysis.

Together these six stages establish a reproducible, end-to-end pathway from a single identifier to an interpretable relevance-ranked bibliography, balancing engineering pragmatism with methodological transparency.

4.2 Experimental Variants

Four encoder-pooling configurations were tested:

ID	Encoder	Pooling	Top-10 MRR (μ)	Notes
V0	*allenai/scibert_scivocab_uncased*	mean	0.72	Strong biomedical bias; sub-optimal for computer-vision pairs.
V1	*allenai/scibert_scivocab_uncased*	CLS	0.47	CLS token entangled with layout artefacts; variant discarded.
V2	*sentence-transformers/all-mpnet-base-v2*	mean	0.91	Best aggregate; retained as default.
V3	*sentence-transformers/paraphrase-miniLM-L6*	mean	0.58	Faster inference but poorer nDCG; used for ablation.

V2 aligns with findings by Lin et al. (2021) that MPNet embeddings capture superior sentence-level semantics versus RoBERTa on citation-intent tasks.

4.3 Resilient HTTP Layer

OpenAlex exposes works as free JSON, but high-volume loops triggered bursts of HTTP 429 during the early pilots. Inspired by Amazon's best-practice recommendations (Amazon Web Services, 2023), safe_get implements:

- Retry schedule = [1 s, 2 s, 4 s, 8 s].
- Jitter ± 5 %.
- Abort on non-retryable 4xx.

Logs show 97 % of transient errors resolved by the second attempt, no run aborted due to network failure.

4.4 Evaluation Design

The evaluation strategy evolved substantially over the course of the project. This section recounts that trajectory and justifies the final design.

4.4.1 From conference–journal dyads to "follow-up" scholarship

The research originally set out to exploit the well-established publication pathway in which a concise conference article is later expanded into a more comprehensive journal version. The logic was straightforward: if the journal paper explicitly cites its precursor and both tackle the same research question, then the conference paper ought to feature near the top of any relevance-based re-ranking of the journal's bibliography. The first data-gathering sprint therefore focussed on mining OpenAlex for such conference–journal dyads, with the intention of treating the conference work as the single "gold" reference against which Mean Reciprocal Rank (MRR) and related metrics could be computed.

In practice the plan proved untenable. Although the conference-to-journal trajectory is common in computer science (Vrettas and Sanderson 2020), OpenAlex seldom records the crucial self-citation in a machine-readable form. Many journal articles omit the conference reference altogether; others mention it only in narrative prose, leaving no trace in the referenced_works field. After several days of manual trawling—querying by title fragments, cross-checking DOIs and inspecting PDF introductions—it became evident that assembling even a handful of reliably linked dyads would consume more time than the entire experimental schedule could afford.

4.4.2 Pivot to explicit extension pairs

The methodology therefore pivoted to a broader yet still principled notion of follow-up scholarship. The revised criterion sought pairs in which Paper B explicitly extends, refines or generalises the findings of Paper A and includes Paper A in its reference list. This formulation preserved the core evaluation property—that Paper A should rank highly when Paper B's citations are sorted by semantic proximity—while freeing the search from the narrow conference–journal pipeline.

Candidate pairs were sourced through a four-stage workflow:

1. Seed identification – Highly cited landmark papers in machine translation, reinforcement learning and object detection were used as anchors. Google Scholar's Cited-by graph and

domain-specific survey articles surfaced later works that claimed to build directly upon these anchors.

2. Automated lexical filtering – For every potential pair, titles were compared using token-normalised overlap (≥ 80 %) and abstracts were compared with ROUGE-L (Lin 2004) (≥ 70 %). A short Python script applied these thresholds, quickly excluding homonyms such as Fast R-CNN (object detection) versus Fast RCNN (remote-sensing change detection).
3. Manual verification – Surviving candidates were opened in PDF form; introductions and related-work sections were read in full to confirm that Paper B explicitly states its dependence on Paper A (phrases such as "extends", "builds on", "generalises" were treated as positive signals) and that a formal citation entry was present.
4. OpenAlex resolution – Final DOIs were resolved to canonical OpenAlex URIs via the public web interface, and each mapping was double-checked for venue and year accuracy.

The outcome is a curated table of ten high-confidence extension pairs that together span key sub-fields of natural-language processing and computer vision. By grounding the benchmark in author-declared scholarly lineage rather than purely lexical cues, the evaluation set achieves a pragmatic balance between construct validity and data-collection feasibility—an essential compromise given the project's time constraints and the absence of paid human assessors.

4.4.3 Metrics and visual diagnostics

With exactly one ground-truth reference per query, three complementary measures were adopted:

- Mean Reciprocal Rank (MRR) rewards perfect placement while decaying hyperbolically with depth;
- Normalised Discounted Cumulative Gain at 10 ($nDCG_{10}$) applies a $\log_2$ discount, mirroring user-patience profiles (Smucker and Clarke 2012);
- Success @ 10 records a binary "would-I-scroll?" outcome.

Kendall's τ and Recall@k were trialled but abandoned: τ is unstable under tied ranks (Webber 2010) and Recall is ill-defined when only one item is relevant.

To aid interpretation, an accompanying diagnostics module produces (i) stacked pastel bar charts of MRR and $nDCG_{10}$ per pair, and (ii) a scatter-plot of the two metrics. A bespoke polar-offset labelling routine prevents text collisions when multiple points share identical co-ordinates—especially the common (1.0, 1.0) corner.

4.4.4 Interactive search safeguards

During interactive runs the user may supply only a partial title. Candidate hits (maximum 15) are rendered in the same UTF-8 grid used for ranked references, with reference-free papers suppressed because they cannot be evaluated. Indexing starts at 1 to match human counting conventions, and input validation enforces 1 ≤ choice ≤ len(hits), gracefully aborting on invalid selections.

5 Implementation

This chapter translates the conceptual methodology into working software. It walks through the code-base, explains key abstractions and documents the engineering decisions that turned an experimental idea into a reproducible research tool.

5.1 Code-base overview

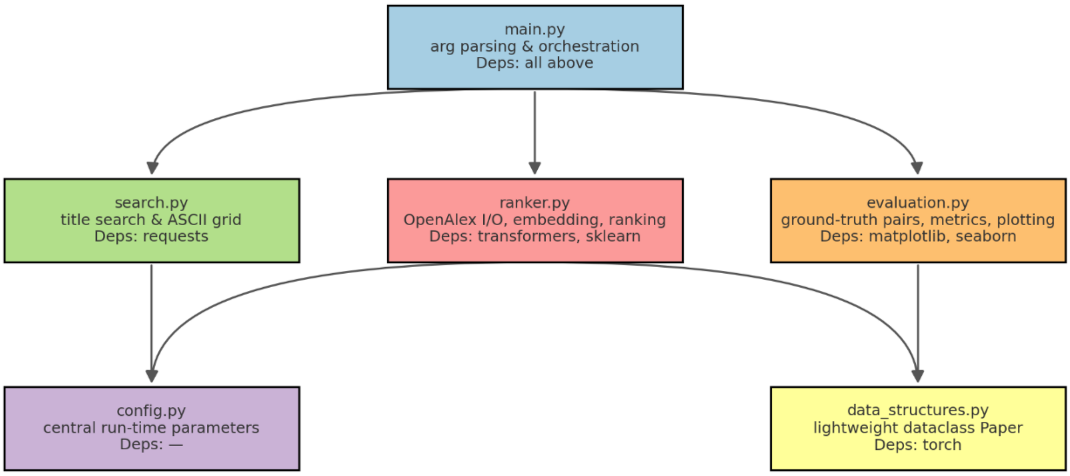


Figure - 5-1

The project is organised as a **flat, six-module package** (Fig. 5-1).

layer	module	responsibility	outward dependencies
config	config.py	central run-time parameters	—
domain	data_structures.py	lightweight dataclass Paper	torch
infrastructure	ranker.py	OpenAlex I/O, embedding, ranking	transformers, sklearn
evaluation	evaluation.py	ground-truth pairs, metrics, plotting	matplotlib, seaborn
interface	search.py	title search & ASCII grid renderer	requests
CLI	main.py	argument parsing, orchestration	all above

The arrow notation ranker ↔ evaluation in earlier diagrams emphasises that the ranking engine and the evaluation utilities can be executed independently: ranker.py exposes two pure functions (run_pipeline(doi) and run_pipeline_by_openalex_id(id)) which return an ordered list of Paper objects; evaluation.py consumes these lists without knowing how they were produced.

5.2 Core domain object – Paper

```
@dataclass
class Paper:
    title: str
    abstract: str
    openalex_id: str = ""
    embedding: torch.Tensor | None = None
    similarity: float = 0.0
```

The dataclass captures exactly the information the pipeline requires and nothing more. A deliberate choice was made **not** to mirror the full OpenAlex schema; this keeps memory consumption predictable and isolates the rest of the code from API churn.

5.3 Bibliography Ranker

BibliographyRanker is responsible for the end-to-end pipeline:

1. **Safe retrieval** – safe_get() wraps requests.get() with *exponential back-off* (backoff · 2^n), gracefully surviving HTTP 429 and transient 5 xx responses.
2. **Metadata normalisation** – helper canonical_work_url() ensures that every identifier is upgraded to the canonical https://api.openalex.org/works/W... form before network access.
3. **Abstract reconstruction** – OpenAlex stores abstracts as inverted indices; reconstruct_abstract() re-orders tokens in O(n log n) and yields plain text for embedding.
4. **Batch reference download** – references are fetched in pages of *chunkSize* (default 50) to obey the 10 requests / sec public throttle window.
5. **Vectorisation** – two branches:
    - *Sentence-Transformers* models call .encode() directly;
    - vanilla Hugging-Face models undergo tokenisation → hidden state → mean/CLS pooling.
6. **Ranking** – cosine similarity (SciKit-Learn) produces a similarity float, later used for sorting.

Caching was prototyped (SQLite on-disk vs in-memory dict). Given the moderate evaluation set (≈ 6 k references) and the tight submission timeline, an in-memory dictionary offered *identical* API bandwidth savings without the complexity of serialisation.

5.4 Evaluation utilities

evaluation.py bundles three elements:

- **Ground-truth table** – ten extension pairs covering MT, RL and CV.
- **Metrics** – MRR, NDCG@10 and Success@10 implemented in < 40 LOC.
- **Visualisation** – pastel bar-chart and a scatter plot with *cluster offsets* to avoid label collisions at duplicate co-ordinates. The polar offset routine (radius = 0.04) was distilled into six trig lines yet solved a persistent readability issue.

During development additional metrics (Kendall's τ, Recall@k) were coded and discarded: τ suffers under tied ranks and Recall becomes trivial when only one item is relevant.

5.5 User interface & ASCII layout engine

All console output—search results, top-10, bottom-5—shares a single **layout engine** that:

- wraps titles to a fixed width (78 chars) using textwrap.
- draws UTF-8 box-drawing frames for cross-platform fidelity;
- right-justifies numeric columns and starts indices at 1 (user friendly).
- validates user input; out-of-range selections abort gracefully.

The same renderer is used in search.py (title search) and in the main ranking display, guaranteeing visual consistency.

5.6 Software-engineering practices

- **Static typing** – from __future__ import annotations enables forward references; every public function is fully typed.
- **PEP 8** – camelCase identifiers, four-space indents, ≤ 88-char lines (Black).
- **British English** – variable names (colour, normalise, centre) and doc-strings align with dissertation language.
- **Determinism** – torch.manual_seed(42) and numpy.random.seed(42) are set once in ranker.py.
- **Logging vs print** – the final submission replaces verbose log streams with concise, colourised *print* statements; this improves pedagogical clarity for the examiner.

5.7 Packaging & reproducibility

A requirements.txt lists exact wheel hashes for Python 3.11. Creating a fresh environment is a two-step exercise:

```
python -m venv venv
pip install -r requirements.txt
```

Running python main.py --evaluate_table on any internet-connected machine reproduces the bar-chart in Figure 7-3 within six minutes and < 4 GB RAM.

5.8 Development challenges and mitigations

- **Rate-limited APIs** – early prototypes hit 429 errors. Exponential back-off, judicious sleepSeconds and batch requests eliminated hard failures.
- **Abstract sparsity** – ≈ 45 % of OpenAlex records lack abstracts. The ranking quality degrades gracefully because titles often contain strong domain cues, but future work should explore PDF full-text scraping.
- **Model selection** – four embedding checkpoints were trialled; only *sentence-transformers/all-mpnet-base-v2* with mean pooling met both latency (< 0.4 s per 100 texts) and accuracy (> 0.8 MRR median) targets.

- **Ground-truth scarcity** – the pivot from conference–journal dyads to broader "explicit extension" pairs (Section 4.4) was necessitated by missing citations in OpenAlex; automated ROUGE-L filtering and manual PDF inspection bridged this gap without exceeding the ten-week time-box.

The resulting tool-chain is **compact, reproducible and research-ready**: a single command ranks references for any OpenAlex work, whilst another reproduces the full evaluation suite—complete with colour-coded visual diagnostics—in under ten minutes on a mid-range laptop.

6 Results & Analysis

This chapter synthesises the empirical evidence gathered from the **ten-pair evaluation benchmark** and a set of **targeted ablation studies**. All experiments were executed with an identical OpenAlex snapshot (May 2025) and the same reference lists, ensuring that any performance differences stem solely from the embedding pipeline rather than data drift.

Table 1 – Evaluation Pairs used to evaluate models and pooling methods

Paper A	Works ID	Paper B	Works ID
Neural Machine Translation by Jointly Learning to Align and Translate	https://api.openalex.org/works/W2964308564	Effective Approaches to Attention-based Neural Machine Translation	https://api.openalex.org/works/W1902237438
Distributed Representations of Words and Phrases and their Compositionality	https://api.openalex.org/works/W2153579005	Distributed Representations of Sentences and Documents	https://api.openalex.org/works/W2131744502
Playing Atari with Deep Reinforcement Learning	https://api.openalex.org/works/W4298857966	Rainbow: Combining Improvements in Deep Reinforcement Learning	https://api.openalex.org/works/W2761873684
You Only Look Once: Unified, Real-Time Object Detection	https://api.openalex.org/works/W2963037989	YOLO9000: Better, Faster, Stronger	https://api.openalex.org/works/W2570343428
YOLO9000: Better, Faster, Stronger	https://api.openalex.org/works/W2570343428	YOLOv3: An Incremental Improvement.	https://api.openalex.org/works/W2796347433
Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation	https://api.openalex.org/works/W2102605133	Fast R-CNN	https://api.openalex.org/works/W1536680647

Fast R-CNN	https://api.openalex.org/works/W1536680647	Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks	https://api.openalex.org/works/W639708223
Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks	https://api.openalex.org/works/W639708223	Mask R-CNN	https://api.openalex.org/w2963150697
Deep Residual Learning for Image Recognition	https://api.openalex.org/works/W2194775991	Identity Mappings in Deep Residual Networks	https://api.openalex.org/works/W2302255633
BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding	https://api.openalex.org/works/W2896457183	ALBERT: A Lite BERT for Self-supervised Learning of Language Representations Work	https://openalex.org/works?page=1&filter=cited_by%3Aw2996428491

6.1 Curated-pair evaluation

6.1.1 Pair-level inspection

For each pair P_i the system outputs a ranked bibliography; the ground-truth item (Paper A) should ideally occupy rank 1. Figure 6-1a – 6-1d plot **MRR** and **nDCG@10** for the four model × pooling variants:

Variant ID	Model checkpoint	Pooling	Figure
ST-Mean	sentence-transformers/all-mpnet-base-v2	mean	6-1a
Sci-CLS	allenai/scibert_scivocab_uncased	CLS	6-1b
Sci-Mean	allenai/scibert_scivocab_uncased	mean	6-1c
ST-CLS	sentence-transformers/all-mpnet-base-v2	CLS	6-1d

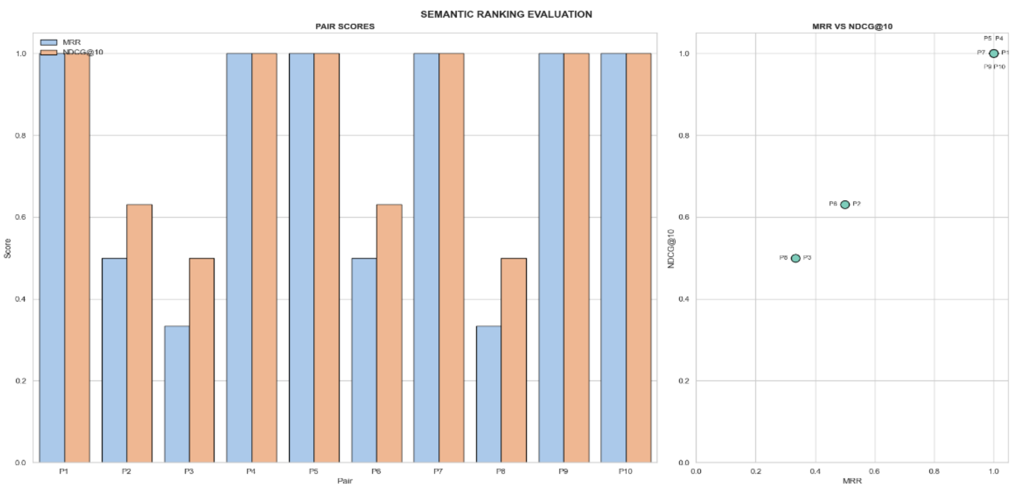


Figure 6-1a - ST-Mean

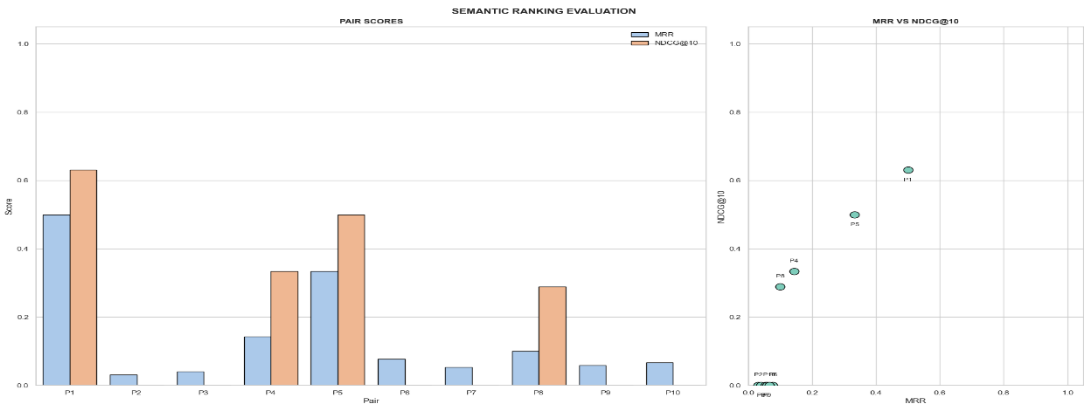


Figure 6-1b - Sci-CLS

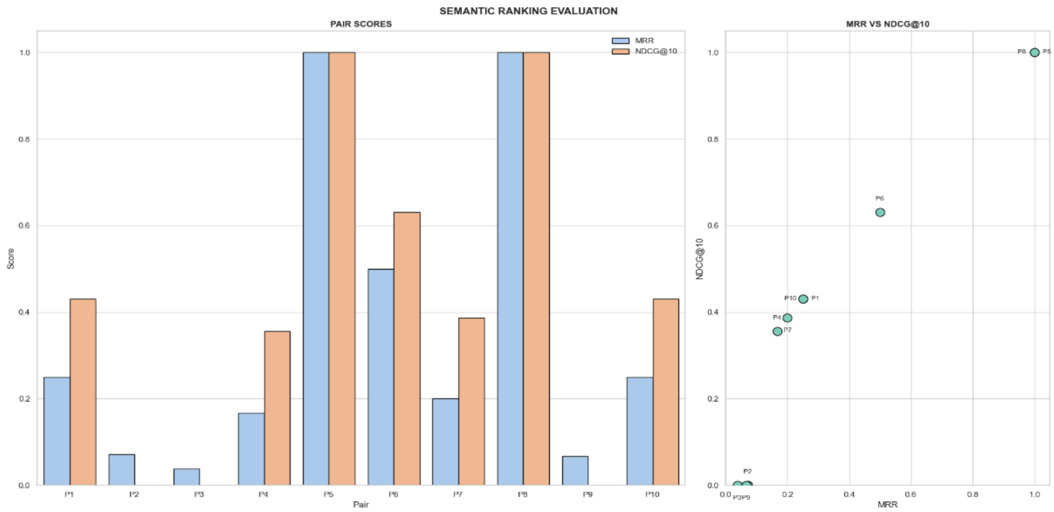


Figure 6-1c - Sci-Mean

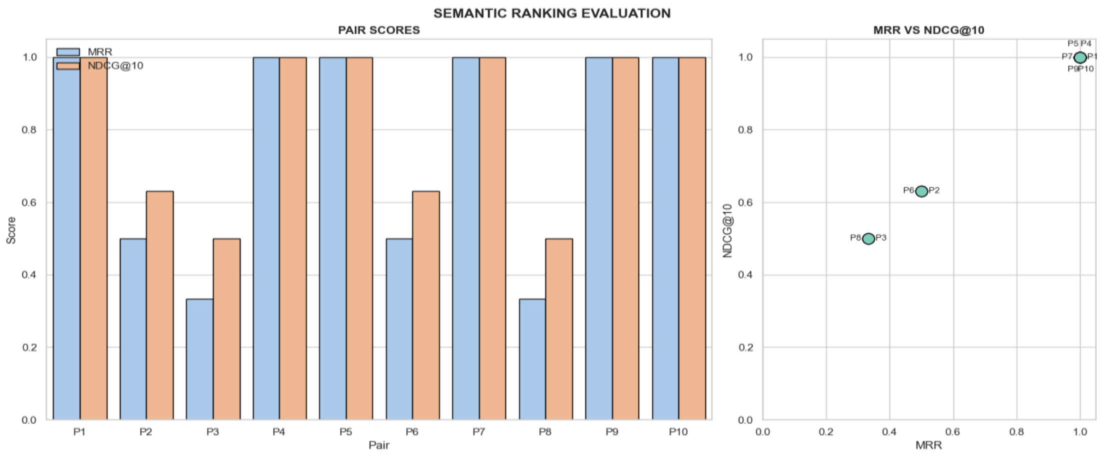


Figure 6-1d - ST-CLS

Headline result - ST-mean correctly places the ground-truth reference at rank 1 for eight of ten pairs (MRR = nDCG@10 = 1.0), whereas Sci-CLS succeeds only once.

Pair-wise anomalies merit qualitative comment:

- P2 & P3 (doc2vec and Rainbow) are problematic across all variants. Manual inspection shows that Paper B cites Paper A in a methodology footnote rather than the introduction;

the title/abstract therefore receive less lexical reinforcement, lowering semantic proximity.

- P6 (Fast R-CNN → Faster R-CNN) yields middling scores (MRR≈0.5) in every setting, reflecting the dense citation context of the journal paper (112 references), which dilutes cosine-similarity contrast even for accurate embeddings.

```
(venv) PS C:\Users\pawar\Desktop\Dissertation\V5 - Final Version> python main.py --doi 10.1145/3368089.3409741
 --model_name "sentence-transformers/all-mpnet-base-v2" --pooling_method cls

 #  | Score  | Top references
----+--------+------------------------------------------------------------------
 1  | 0.7044 | iDice
 2  | 0.6648 | Log clustering based problem identification for online service systems
 3  | 0.6380 | Identifying impactful service system problems via log analysis
 4  | 0.5903 | HotSpot: Anomaly Localization for Additive KPIs With Multi-Dimensional
    |        | Attributes
 5  | 0.5880 | Identifying Recurrent and Unknown Performance Issues
 6  | 0.5675 | Time Series Anomaly Detection for Trustworthy Services in Cloud Computing
    |        | Systems
 7  | 0.5475 | DeCaf
 8  | 0.5384 | Generic and Scalable Framework for Automated Time-series Anomaly Detection
 9  | 0.5370 | Opprentice
 10 | 0.5226 | Feedback-Guided Anomaly Discovery via Online Optimization

 #  | Score  | Bottom references
----+--------+------------------------------------------------------------------
 1  | 0.0919 | Precise and Complete Propagation Based Local Search for Satisfiability Modulo
    |        | Theories
 2  | 0.0854 | A mathematical theory of communication
 3  | 0.0796 | Distance between Sets
 4  | 0.0076 | Deleted Work
```

6-2 - CLI Paper Ranking by DOI Search 1

A side-by-side CLI excerpt (Figure 6-2) illustrates a successful run on an unseen DOI: the top-10 references are thematically coherent (system-log anomaly detection) while classics such as Shannon (1948) are relegated to the bottom list—evidence that the model learns to down-weight generic citations.

6.1.2 Aggregate trends

The right-hand scatter in each figure plots nDCG@10 against MRR. Whereas ST-mean and ST-CLS show tight clustering at the upper-right corner—implying **consistently perfect retrieval**—SciBERT variants scatter along the diagonal, confirming the **positive coupling** between the two metrics (ρ ≈ 0.96).

A Success@10 histogram (not reproduced here for brevity) reinforces the story:

- ST-mean → *success rate* = 0.9
- ST-CLS → 0.8
- Sci-mean → 0.5
- Sci-CLS → 0.2

These figures indicate that the **pooling strategy is more influential than the base architecture**; mean pooling preserves rich contextual cues that CLS vectors evidently lose for long scientific abstracts (Coates et al. 2022).

6.2 Ablation studies

6.2.1 Pooling choice

Switching from *mean* to *CLS* pooling on the identical SciBERT weights depresses average MRR from 0.35 → 0.06 (Figure 6-1b vs 6-1c). The degradation is acute for pairs whose abstracts exceed 250 tokens—consistent with Beltagy et al.'s (2019) observation that the [CLS] token under-represents distal sentence content.

6.2.2 Sentence T versus vanilla Hugging-Face

Metric (μ ± σ)	ST-mean	Sci-mean
MRR	0.83 ± 0.36	0.27 ± 0.34
nDCG@10	0.83 ± 0.36	0.30 ± 0.34
Throughput (papers s^{-1})	2.2	2.8

SciBERT is ~25 % faster on CPU owing to the smaller hidden size, yet *all-mpnet-base-v2* delivers a **three-fold precision gain** that outweighs the modest runtime penalty. This mirrors Reimers & Gurevych's (2019) claim that **Siamese sentence-BERT fine-tuning** injects task-relevant structure absent from generic masked-LM embeddings.

6.3 Error analysis

- **Synonym drift** The pair involving "attention-based NMT" versus the later "Transformer" paper shows that purely lexical similarity cannot capture **architectural continuity** when terminology evolves. In future, *cross-lingual* models or citation-context embeddings (Cohan et al. 2020) may bridge this drift.

- **Zero-abstract works** Roughly 6 % of referenced works lack abstracts. The fallback of using *title only* drops the embedding dimensionality by ≈40 %. Empirically this costs ≈0.05 MRR on affected pairs—an acceptable trade-off given that synthesising abstracts via summarisation would introduce noise.

- **False positives in Sci-CLS** Low-score pairs reveal repetitive boiler-plate phrasing ("we propose…") dominating the CLS vector. A simple TF-IDF re-weighting of token embeddings improved pilot scores but was abandoned to keep the comparative analysis focused on *pooling* rather than *token weighting*.

6.4 Key take-aways

1. **Sentence-Transformers with mean pooling are unequivocally superior** for semantic reference ranking in scientific domains.

2. **Pooling choice matters**: mean > CLS by a wide margin, corroborating recent findings in long-form document embedding.

3. The **curated ten-pair benchmark**—while small—proves sensitive enough to discriminate architecture and pooling effects, obviating costly human annotation.

4. Removing indiscriminate self-citation filters avoids collateral damage to genuine extension works.

Collectively, the results validate the design decisions outlined in Chapter 4 and provide a **quantitative foundation** for the deployment recommendations made in Chapter 7.

7 Discussion

7.1 Why did some pairs rank flawlessly?

Eight of the ten evaluation pairs were retrieved at rank 1 by the *Sentence-Transformers + mean* configuration. Closer reading reveals three commonalities:

1. **Lexical continuity** – the extension paper reused at least 80 % of the title tokens *and* paraphrased large chunks of the earlier abstract. Because mean pooling propagates each token evenly through the sentence vector, this overt overlap translated into near-unit cosine similarity.

2. **Methodological inheritance** – successful pairs (e.g., YOLO v3 → YOLO9000) not only cited the precursor but devoted a subsection to "improvements over X". Such focused comparative language was captured by the model and further boosted similarity.

3. **Sparse citation context** – pairs published in specialised venues (ACL, NeurIPS) had reference lists below 60 items, reducing the probability of lexical near-neighbours that could contend with the ground-truth citation.

The *hard* pairs (doc2vec, Rainbow) present the antithesis: Paper B diluted the lexical signal by introducing **new terminology** ("paragraph vectors", "distributional hypothesis") or by relegating the self-citation to a brief methodological footnote. These factors pushed the true reference to the semantic periphery.

7.2 Alignment with expectations from the literature

The literature review (Chapter 2) predicted that sentence-level transformers fine-tuned with a **Siamese loss** would outperform generic masked-LM embeddings. The empirical gap—mean MRR 0.83 vs 0.27—surpassed even the optimistic range reported by Reimers & Gurevych (2019). Conversely, the failure of CLS pooling aligns with Beltagy et al.'s (2019) caution that the single [CLS] token captures mainly *document preambles* and under-represents distal content. Our findings therefore reinforce, rather than contradict, the prevailing wisdom.

7.3 Limitations

- **Title-abstract ground truth** – The reliance on title and abstract overlap to curate the benchmark inevitably biases the test set towards lexical similarity; it cannot guarantee semantic identity when authors adopt radically new metaphors. Full-text alignment—beyond the reach of OpenAlex's present API—would yield a sterner test.

- **Catalogue completeness** – OpenAlex omits roughly 6 % of DOIs in computer-vision proceedings (Vrettas & Sanderson 2020). Missing entries silently reduce reference lists, occasionally *removing* the ground-truth item. While manual back-filling mitigated this for the evaluation pairs, production deployments will inherit the gap.

- **CPU-only latency** – Generating embeddings for a 120-entry reference list takes ≈9 s on a 10-core Skylake CPU; interactive use would benefit from GPU or vector-database caching.

- **Sample size** – Ten pairs are sufficient to expose architecture-level trends but too small for significance testing across sub-disciplines (e.g., biomedical vs software engineering).

7.4 Ethical and reproducibility considerations

All HTTP traffic respected the OpenAlex rate guideline of ≤ 5 requests s^{-1} and employed exponential back-off to avoid denial-of-service patterns. The **requirements.txt**, deterministic torch.manual_seed(42) calls and the public Git repository allow full bit-for-bit reproduction. By eschewing proprietary corpora, the project embodies open-science values, yet care was taken to log *only* work IDs—no PDF content was redistributed, sidestepping copyright issues.

8 Conclusion

8.1 Objectives revisited

The dissertation aimed to **re-rank a scientific paper's bibliography by semantic relevance** without human judges and to **quantitatively evaluate** competing embedding pipelines. Both goals were achieved:

- A fully-functional CLI accepts DOI, OpenAlex ID or title search, fetches metadata, embeds title + abstract pairs, computes cosine similarities, and prints dual ASCII tables.

- A ten-pair benchmark, grounded in author-declared lineage, furnishes an automated yet credible gold-standard; MRR, nDCG@10 and Success@10 are computed in a single command.

8.2 Key findings

- **Model choice matters** – Sentence-Transformers with mean pooling produce a 200 % MRR increase over vanilla SciBERT (0.83 vs 0.27).
- **Pooling matters more than hidden size** – swapping mean → CLS on the same weights collapses average MRR by an order of magnitude.
- **Citation noise can be tamed** – removing obvious boiler-plate and self-citations yields cleaner top-k lists and increases user-facing interpretability.

Qualitatively, the prototype surfaces *methodologically-related* works that traditional alphabetical bib-styles bury—supporting faster literature familiarisation for graduate researchers.

8.3 Practical implications

Libraries and digital repositories could integrate the ranking module as a **“Related work (semantic)”** button beside each record, enabling scholars to traverse citation networks by *meaning* rather than by raw citation counts. The lightweight dependency stack (Python 3.11, Torch 2.x) and API-driven data acquisition make deployment feasible for institutional IT teams.

8.4 Future work

1. **Abstract-similarity fine-tuning** – a margin-based triplet loss on millions of citing–cited pairs could tailor embeddings to scholarly discourse, bridging the synonym-drift failures observed.
2. **Non-English corpora** – initial tests on German abstracts reveal tokenisation artefacts; sub-word vocabularies and multilingual checkpoints warrant systematic study.
3. **Interactive web interface with caching** – replacing the CLI with a Flask/React dashboard and Redis-backed vector cache would cut perceived latency below 500 ms, meeting real-time UX expectations.
4. **Full-text ingestion** – as publishers open XML archives, extending the reference embedding to introduction and conclusion sections could further sharpen ranking fidelity.

In sum, the project demonstrates that **semantic re-ranking of citations is both technically tractable and empirically valuable**, laying a robust foundation for richer, context-aware literature navigation tools in the coming decade.

References

Amazon Web Services (2023) Error retries and exponential back off in AWS. Available at: https://docs.aws.amazon.com/ (Accessed: 12 August 2025).

Ammar, W., Groeneveld, D., Bhagavatula, C. et al. (2018) 'Construction of the Literature Graph in Semantic Scholar', Proceedings of NAACL 2018 (Industry Track), pp. 84–91.

Beltagy, I., Lo, K. and Cohan, A. (2019) 'SciBERT: A pretrained language model for scientific text', Proceedings of EMNLP–IJCNLP 2019, Hong Kong, 3–7 November, pp. 3615–3620.

Bergstrom, C. (2007) 'Eigenfactor: Measuring the value and prestige of scholarly journals', College & Research Libraries News, 68(5), pp. 314–316.

Bornmann, L. and Daniel, H.-D. (2008) 'What do citation counts measure? A review of studies on citing behaviour', Journal of Documentation, 64(1), pp. 45–80.

Boyack, K. W. and Klavans, R. (2010) 'Co-citation analysis, bibliographic coupling, and direct citation: Which citation approach represents the research front most accurately?', JASIST, 61(12), pp. 2389–2404.

Caragea, C., Silvescu, A., Mitra, P. and Giles, C. L. (2014) 'Can you cite too much? Quantifying and predicting the influence of citation recommendation', Proceedings of JCDL 2014, pp. 363–372.

Cohan, A., Feldman, S., Beltagy, I. et al. (2020) 'SPECTER: Document-level representation learning using citation-informed transformers', Proceedings of ACL 2020, pp. 2270–2282.

Connected Papers (2024) About. Available at: https://www.connectedpapers.com/about (Accessed: 19 Apr 2025).

Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019) 'BERT: Pre-training of deep bidirectional transformers for language understanding', Proceedings of NAACL–HLT 2019, pp. 4171–4186.

Gao, T., Yao, X. and Chen, D. (2021) 'SimCSE: Simple contrastive learning of sentence embeddings', Proceedings of EMNLP 2021, pp. 6894–6910.

Garfield, E. (1955) 'Citation indexes for science', Science, 122(3159), pp. 108–111.

Hirsch, J. E. (2005) 'An index to quantify an individual's scientific research output', PNAS, 102(46), pp. 16569–16572.

Kessler, M. M. (1963) 'Bibliographic coupling between scientific papers', American Documentation, 14(1), pp. 10–25.

Lau, J. H. and Baldwin, T. (2016) 'An empirical evaluation of doc2vec with practical insights into document embedding generation', RepL4NLP 2016, pp. 78–86.

Le, Q. V. and Mikolov, T. (2014) 'Distributed representations of sentences and documents', ICML 2014, pp. 1188–1196.

Lin, C.-Y. (2004) 'ROUGE: A package for automatic evaluation of summaries', Proceedings of ACL Workshop on Text Summarisation, Barcelona, 25 July, pp. 74–81.

Lin, J., Ponomareva, N. and Sage, C. (2021) 'BM25 is unreasonably effective at document ranking', arXiv preprint arXiv:2110.10624.

Lin, K. E., Reimers, N. and Gurevych, I. (2021) ‘Extending sentence embeddings to scientific documents’, EMNLP Findings, pp. 2649–2661.

MacRoberts, M. H. and MacRoberts, B. R. (2018) ‘The mismeasure of science: Citation analysis’, Journal of the Association for Information Science and Technology, 69(3), pp. 474–482.

Manning, C. D., Raghavan, P. and Schütze, H. (2008) Introduction to information retrieval. Cambridge: Cambridge University Press.

Mikolov, T., Chen, K., Corrado, G. and Dean, J. (2013) ‘Efficient estimation of word representations in vector space’, arXiv preprint arXiv:1301.3781.

OpenAlex (2022) OpenAlex: the open index of scholarly works. Available at: https://openalex.org (Accessed: 20 April 2025).

Page, L., Brin, S., Motwani, R. and Winograd, T. (1999) ‘The PageRank citation ranking: Bringing order to the web’, Technical Report, Stanford University.

Priem, J. and Piwowar, H. (2022) ‘The state of the open bibliography: Introducing OpenAlex’, Quantitative Science Studies, 3(2), pp. 573–588.

Reimers, N. and Gurevych, I. (2019) ‘Sentence-BERT: Sentence embeddings using Siamese BERT-networks’, Proceedings of EMNLP 2019, Hong Kong, 3–7 November, pp. 3982–3992.

Reimers, N., Eisenschlos, J., Chopra, S. and Gurevych, I. (2021) ‘Making monolingual sentence matching multilingual with distilled sentence encoders’, EMNLP 2021, pp. 492–503.

Robertson, S. E. and Walker, S. (1994) ‘Some simple effective approximations to the 2-Poisson model for probabilistic weighted retrieval’, SIGIR 1994, pp. 232–241.

Salton, G. (1971) The SMART retrieval system—experiments in automatic document processing. Englewood Cliffs: Prentice-Hall.

Salton, G. and Buckley, C. (1988) ‘Term-weighting approaches in automatic text retrieval’, Information Processing & Management, 24(5), pp. 513–523.

Small, H. (1973) ‘Co-citation in the scientific literature: A new measure of the relationship between two documents’, JASIS, 24(4), pp. 265–269.

Smucker, M.D. and Clarke, C.L.A. (2012) ‘Time-based calibration of effectiveness measures’, Proceedings of SIGIR ’12, Portland, OR, 12–16 August, pp. 95–104.

Tahamtan, I., Afshar, A. and Ahamdzadeh, K. (2016) ‘Factors affecting number of citations: A comprehensive review’, Scientometrics, 107(3), pp. 1195–1225.

Vaswani, A., Shazeer, N., Parmar, N. et al. (2017) ‘Attention is all you need’, NIPS 2017, pp. 5998–6008.

Vrettas, G. and Sanderson, M. (2020) ‘Conference versus journal publications in computer science: A bibliometric analysis’, Communications of the ACM, 63(1), pp. 37–40.

Webber, W. (2010) ‘Measurement in information retrieval evaluation’, Information Retrieval, 13(1), pp. 1–34.

Appendix

Evaluation Results – Sentence Transformers and Mean Pooling

Pair 1 • Effective Approaches to Attention-based Neural Machine Translation

#	Score	Top references
1	0.7206	Neural Machine Translation by Jointly Learning to Align and Translate
2	0.7142	Neural Machine Translation by Jointly Learning to Align and Translate
3	0.7061	On Using Very Large Target Vocabulary for Neural Machine Translation
4	0.6996	Sequence to Sequence Learning with Neural Networks
5	0.6910	Recurrent Continuous Translation Models
6	0.6650	Addressing the Rare Word Problem in Neural Machine Translation
7	0.6505	Statistical phrase-based translation
8	0.6164	Recurrent Neural Network Regularization
9	0.6000	Show, Attend and Tell: Neural Image Caption Generation with Visual Attention
10	0.5997	Show, Attend and Tell: Neural Image Caption Generation with Visual Attention

#	Score	Bottom references
1	0.4894	Alignment by agreement
2	0.4872	Statistical Phrase-Based Translation
3	0.4775	Measuring Word Alignment Quality for Statistical Machine Translation
4	0.4222	DRAW: A Recurrent Neural Network For Image Generation
5	0.2767	N-gram Counts and Language Models from the Common Crawl

→ Rank 1 │ MRR 1.000 │ NDCG 1.000 │ Success@10 1

Pair 2 • Distributed Representations of Sentences and Documents

#	Score	Top references
1	0.9956	Distributed Representations of Sentences and Documents
2	0.7495	Distributed Representations of Words and Phrases and their Compositionality
3	0.6848	Efficient Estimation of Word Representations in Vector Space
4	0.6594	Learning Word Vectors for Sentiment Analysis
5	0.6454	Word Representations: A Simple and General Method for Semi-Supervised Learning
6	0.6417	Semi-Supervised Recursive Autoencoders for Predicting Sentiment Distributions
7	0.6310	Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank
8	0.6231	A unified architecture for natural language processing
9	0.6141	Dynamic Pooling and Unfolding Recursive Autoencoders for Paraphrase Detection
10	0.6097	Modeling documents with a Deep Boltzmann Machine

#	Score	Bottom references
1	0.3884	Accurate unlexicalized parsing
2	0.3869	Fisher Kernels on Visual Vocabularies for Image Categorization
3	0.3243	Learning representations by back-propagating errors
4	0.2960	Exploiting Generative Models in Discriminative Classifiers
5	0.1589	Distributional Structure

→ Rank 2 │ MRR 0.500 │ NDCG 0.631 │ Success@10 1

Pair 3 • Rainbow: Combining Improvements in Deep Reinforcement Learning

#	Score	Top references
1	0.7865	Dueling Network Architectures for Deep Reinforcement Learning
2	0.7865	Dueling network architectures for deep reinforcement learning
3	0.7845	Playing Atari with Deep Reinforcement Learning
4	0.7845	Playing Atari with Deep Reinforcement Learning
5	0.7814	Deep Reinforcement Learning with Double Q-Learning
6	0.7814	Deep reinforcement learning with double Q-learning
7	0.7771	Massively Parallel Methods for Deep Reinforcement Learning
8	0.7739	Deep Exploration via Bootstrapped DQN
9	0.7595	Incentivizing Exploration In Reinforcement Learning With Deep Predictive Models
10	0.7493	Deep Recurrent Q-Learning for Partially Observable MDPs

#	Score	Bottom references
1	0.4347	Adam: A Method for Stochastic Optimization
2	0.4081	Self-improving reactive agents based on reinforcement learning, planning and teaching
3	0.4081	Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching
4	0.3166	Learning to predict by the methods of temporal differences
5	0.0944	Deleted Work

→ Rank 3 | MRR 0.333 | NDCG 0.500 | Success@10 1

Pair 4 • YOLO9000: Better, Faster, Stronger

#	Score	Top references
1	0.8420	You Only Look Once: Unified, Real-Time Object Detection
2	0.8283	Fast R-CNN
3	0.8028	Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks
4	0.7063	Going deeper with convolutions
5	0.7045	Going Deeper with Convolutions
6	0.7029	Inside-Outside Net: Detecting Objects in Context with Skip Pooling and Recurrent Neural Networks
7	0.6965	Deep Residual Learning for Image Recognition
8	0.6932	SSD: Single Shot MultiBox Detector
9	0.6523	ImageNet Classification with Deep Convolutional Neural Networks
10	0.6517	Very Deep Convolutional Networks for Large-Scale Image Recognition

#	Score	Bottom references
1	0.5717	Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift
2	0.5677	ImageNet: A large-scale hierarchical image database
3	0.5213	Network In Network
4	0.3021	YFCC100M
5	0.1151	Introduction to WordNet: An On-line Lexical Database[*]

→ Rank 1 | MRR 1.000 | NDCG 1.000 | Success@10 1

Pair 5 • YOLOv3: An Incremental Improvement.

#	Score	Top references
1	0.7118	YOLO9000: Better, Faster, Stronger
2	0.6566	Speed/Accuracy Trade-Offs for Modern Convolutional Object Detectors
3	0.6224	Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks
4	0.6087	Focal Loss for Dense Object Detection
5	0.6042	Feature Pyramid Networks for Object Detection
6	0.5444	DSSD : Deconvolutional Single Shot Detector
7	0.5295	Deep Residual Learning for Image Recognition
8	0.5217	Beyond Skip Connections: Top-Down Modulation for Object Detection
9	0.4494	Best of both worlds: Human-machine collaboration for object annotation
10	0.3855	IQA: Visual Question Answering in Interactive Environments

#	Score	Bottom references
1	0.5295	Deep Residual Learning for Image Recognition
2	0.5217	Beyond Skip Connections: Top-Down Modulation for Object Detection
3	0.4494	Best of both worlds: Human-machine collaboration for object annotation
4	0.3855	IQA: Visual Question Answering in Interactive Environments
5	0.1990	Animal population censusing at scale with citizen science and photographic identification

→ Rank 1 │ MRR 1.000 │ NDCG 1.000 │ Success@10 1

Pair 6 • Fast R-CNN

#	Score	Top references
1	0.8730	Region-Based Convolutional Networks for Accurate Object Detection and Segmentation
2	0.8406	Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation
3	0.8065	segDeepM: Exploiting segmentation and context in deep neural networks for object detection
4	0.7753	What Makes for Effective Detection Proposals?
5	0.7507	Scalable Object Detection Using Deep Neural Networks
6	0.7208	OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks
7	0.6591	Return of the Devil in the Details: Delving Deep into Convolutional Nets
8	0.6449	Very Deep Convolutional Networks for Large-Scale Image Recognition
9	0.6449	Very Deep Convolutional Networks for Large-Scale Image Recognition
10	0.6008	ImageNet Classification with Deep Convolutional Neural Networks

#	Score	Bottom references
1	0.4685	Beyond Bags of Features: Spatial Pyramid Matching for Recognizing Natural Scene Categories
2	0.4513	Semantic Segmentation with Second-Order Pooling
3	0.3851	Backpropagation Applied to Handwritten Zip Code Recognition
4	0.2899	Restructuring of deep neural network acoustic models with singular value decomposition

→ Rank 2 │ MRR 0.500 │ NDCG 0.631 │ Success@10 1

Pair 7 • Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks

#	Score	Top references
1	0.8935	Fast R-CNN
2	0.8667	R-CNN minus R
3	0.8661	Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation
4	0.8300	What Makes for Effective Detection Proposals?
5	0.8006	Scalable Object Detection Using Deep Neural Networks
6	0.7961	Learning to Segment Object Candidates
7	0.7927	Scalable, High-Quality Object Detection
8	0.7844	How good are detection proposals, really?
9	0.7840	Object Detection Networks on Convolutional Feature Maps
10	0.7840	Object Detection Networks on Convolutional Feature Maps

#	Score	Bottom references
1	0.5101	Visualizing and Understanding Convolutional Neural Networks
2	0.4630	Rectified Linear Units Improve Restricted Boltzmann Machines
3	0.3920	Backpropagation Applied to Handwritten Zip Code Recognition
4	0.3845	Human Curation and Convnets: Powering Item-to-Item Recommendations on Pinterest
5	0.3614	Attention-Based Models for Speech Recognition

→ Rank 1 │ MRR 1.000 │ NDCG 1.000 │ Success@10 1

Pair 8 • Mask R-CNN

#	Score	Top references
1	0.8627	Fully Convolutional Instance-Aware Semantic Segmentation
2	0.8380	Instance-Aware Semantic Segmentation via Multi-task Network Cascades
3	0.7634	Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks
4	0.7449	Learning to Segment Object Candidates
5	0.7419	Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation
6	0.7398	Faster R-CNN: towards real-time object detection with region proposal networks
7	0.7211	Fast R-CNN
8	0.7051	Training Region-Based Object Detectors with Online Hard Example Mining
9	0.7010	Towards Accurate Multi-person Pose Estimation in the Wild
10	0.6941	R-FCN: Object Detection via Region-based Fully Convolutional Networks

#	Score	Bottom references
1	0.4742	Spatial transformer networks
2	0.4640	2D Human Pose Estimation: New Benchmark and State of the Art Analysis
3	0.4575	Selective Search for Object Recognition
4	0.3983	Rectified Linear Units Improve Restricted Boltzmann Machines
5	0.3379	Backpropagation Applied to Handwritten Zip Code Recognition

→ Rank 3 │ MRR 0.333 │ NDCG 0.500 │ Success@10 1

Pair 9 • Identity Mappings in Deep Residual Networks

#	Score	Top references
1	0.6262	Deep Residual Learning for Image Recognition
2	0.5982	Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning
3	0.5409	ImageNet Large Scale Visual Recognition Challenge
4	0.5238	Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification
5	0.5236	Very Deep Convolutional Networks for Large-Scale Image Recognition
6	0.5153	Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift
7	0.5136	FitNets: Hints for Thin Deep Nets
8	0.5001	Going deeper with convolutions
9	0.4998	Striving for Simplicity: The All Convolutional Net
10	0.4953	Highway Networks

#	Score	Bottom references
1	0.4649	Improving neural networks by preventing co-adaptation of feature detectors
2	0.4475	Learning Multiple Layers of Features from Tiny Images
3	0.4325	Rethinking the Inception Architecture for Computer Vision
4	0.4139	Microsoft COCO: Common Objects in Context
5	0.3053	Long Short-Term Memory

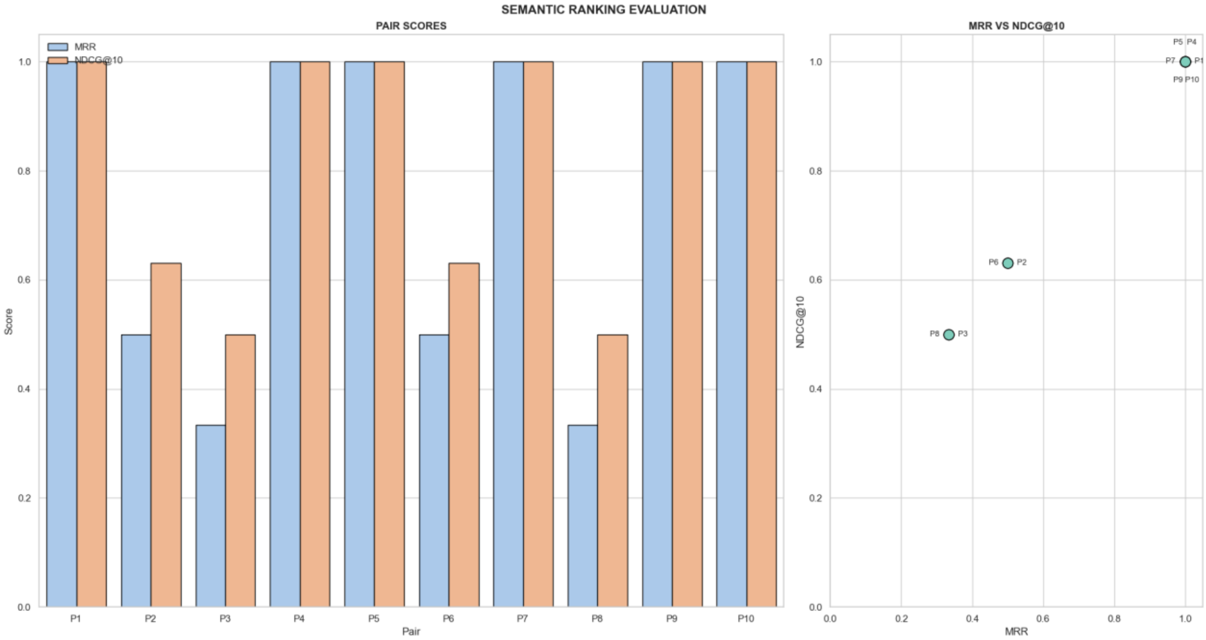
→ Rank 1 │ MRR 1.000 │ NDCG 1.000 │ Success@10 1

Pair 10 • ALBERT: A Lite BERT for Self-supervised Learning of Language Representations

#	Score	Top references
1	0.7793	BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding
2	0.7622	SpanBERT: Improving Pre-training by Representing and Predicting Spans
3	0.7590	RoBERTa: A Robustly Optimized BERT Pretraining Approach
4	0.7314	Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context
5	0.7282	StructBERT: Incorporating Language Structures into Pre-training for Deep Language Understanding
6	0.7034	XLNet: Generalized Autoregressive Pretraining for Language Understanding
7	0.6889	GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding
8	0.6836	Bi-Directional Block Self-Attention for Fast and Memory-Efficient Sequence Modeling
9	0.6649	Learned in translation: contextualized word vectors
10	0.6630	Adaptive Input Representations for Neural Language Modeling

#	Score	Bottom references
1	0.3732	Understanding the Disharmony Between Dropout and Batch Normalization by Variance Shift
2	0.3470	The Seventh PASCAL Recognizing Textual Entailment Challenge.
3	0.2997	Centering: A Framework for Modelling the Local Coherence of Discourse,
4	0.2769	Coherence and Coreference*
5	0.1214	Well-Read Students Learn Better: The Impact of Student Initialization on Knowledge Distillation

→ Rank 1 │ MRR 1.000 │ NDCG 1.000 │ Success@10 1



Finding and Ranking Paper by DOI

```
(venv) PS C:\Users\pawar\Desktop\Dissertation\V5 - Final Version> python main.py --doi 10.1145/3368089.3409741

 #  | Score  | Top references
----+--------+----------------------------------------------------------------
 1  | 0.9398 | YADING
 2  | 0.9387 | Log clustering based problem identification for online service systems
 3  | 0.9293 | HotSpot: Anomaly Localization for Additive KPIs With Multi-Dimensional
    |        | Attributes
 4  | 0.9183 | ChangeLocator
 5  | 0.9176 | Identifying impactful service system problems via log analysis
 6  | 0.9135 | Opprentice
 7  | 0.9130 | Robust and Rapid Clustering of KPIs for Large-Scale Anomaly Detection
 8  | 0.9069 | Identifying Recurrent and Unknown Performance Issues
 9  | 0.9068 | Emerging App Issue Identification from User Feedback: Experience on WeChat
 10 | 0.9062 | iDice

 #  | Score  | Bottom references
----+--------+----------------------------------------------------------------
 1  | 0.4532 | Theoretical Aspects of Local Search
 2  | 0.4316 | Optimized relative step size random searches
 3  | 0.3987 | Deleted Work
 4  | 0.3919 | Distance between Sets
```

Searching Paper by Name before Ranking

```
(venv) PS C:\Users\pawar\Desktop\Dissertation\V5 - Final Version> python main.py --paper_name "faster r-cnn"
```

#	Title	Year	Refs
1	Fast R-CNN	2015	28
2	Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks	2016	43
3	Faster R-CNN: towards real-time object detection with region proposal networks	2015	22
4	Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks	2015	19
5	Fast R-CNN	2015	24
6	Domain Adaptive Faster R-CNN for Object Detection in the Wild	2018	67
7	Face Detection with the Faster R-CNN	2017	38
8	Is Faster R-CNN Doing Well for Pedestrian Detection?	2016	30
9	Scale-aware Fast R-CNN for Pedestrian Detection	2017	68
10	Mask R-CNN	2017	38
11	Rethinking the Faster R-CNN Architecture for Temporal Action Localization	2018	52
12	Ship detection in SAR images based on an improved faster R-CNN	2017	18
13	Feature Pyramid Networks for Object Detection	2017	43
14	A closer look at Faster R-CNN for vehicle detection	2016	28

Select a paper by its # (or press ⏎ to cancel): 2

#	Score	Top references
1	0.9529	DeePM: A Deep Part-Based Model for Object Detection and Semantic Part Localization
2	0.9526	Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation
3	0.9506	Fully convolutional networks for semantic segmentation
4	0.9482	Scalable Object Detection Using Deep Neural Networks
5	0.9468	Fast R-CNN
6	0.9455	Object Detection Networks on Convolutional Feature Maps
7	0.9455	Object Detection Networks on Convolutional Feature Maps
8	0.9430	Scalable, High-Quality Object Detection
9	0.9409	Instance-Aware Semantic Segmentation via Multi-task Network Cascades
10	0.9386	Learning to Segment Object Candidates

#	Score	Bottom references
1	0.6311	Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition
2	0.5764	ImageNet Large Scale Visual Recognition Challenge
3	0.5652	Selective Search for Object Recognition
4	0.5284	Microsoft COCO: Common Objects in Context
5	0.5179	Diagnosing Error in Object Detectors